

SSOT-CENTRIC AGENTIC SOFTWARE ENGINEERING: A KNOWLEDGE-GOVERNED LIFECYCLE FRAMEWORK FOR AI-NATIVE SOFTWARE DEVELOPMENT

Emmanuel Chinyere Echeonwu

Abstract—The rapid development of autonomous software engineering agents and large language models (LLMs) is bringing about a fundamental change in the conception, design, implementation, testing, and maintenance of software systems. Before creating software artefacts, engineers interpret requirements, architectural choices, and technical specifications in traditional software engineering lifecycles that were built around human collaboration. While AI-assisted development tools have shown great promise for increasing developer productivity, current methods often have poor synchronization between requirements, architecture, implementation, and testing; fragmented context management, and limited system understanding.

This ongoing study introduces the SSOT-Centric Agentic Software Engineering Framework, a practical implementation of the SSOT-Centric Agentic Software Engineering and a knowledge-governed lifecycle model with a Single Source of Truth (SSOT) which serves as an intermediary intelligence layer between software engineering artefacts and AI coding agents. The proposed framework combines the engineering artefacts such as Software Requirements Documents (SRDs), Technical Evaluations, Technical Design Documents (TDDs), Architecture Decision Records (ADRs), C4 architecture models, and repository knowledge to form a single project knowledge structure. Unlike traditional implementation-first approaches, in which tasks are directly converted into code, or architecture-first approaches, in which design artefacts primarily guide development, the proposed approach creates a continuous knowledge lifecycle in which engineering artefacts generate an SSOT, the SSOT generates implementation strategies, implementation strategies and the SSOT generate execution plans, and AI agents execute changes under architectural governance.

Keywords—AI-native software engineering, single source of truth, agentic programming

I. INTRODUCTION

The introduction of large language models and AI software agents signals a new development paradigm in which AI systems can participate in tasks previously handled by software developers. Modern AI systems are becoming capable of generating code, analyzing repositories, establishing tests, reviewing implementations, and proposing architectural modifications. Research has shown that LLM-based systems can perform meaningfully in software engineering activities such as code generation, debugging, and automated testing [2], [3]. However, the ability to write code does not always indicate the ability to comprehend the entire engineering context of a software system.

A significant constraint of existing AI-assisted software engineering methodologies is the lack of an architectural and authoritative knowledge model for the system under development. Software projects frequently include several sources of information, such as requirements documents, architecture diagrams, technical specifications, source code, tests, and deployment configurations. These sources frequently become inconsistent over time because implementation changes are not always reflected in design artefacts. This problem, also known as architectural drift and documentation erosion, poses challenges for both human

engineers and AI agents. Existing research provides significant advances in AI-assisted programming, autonomous agents, and software architecture management. However, there is a knowledge gap between engineering knowledge representation and software execution.

Current approaches generally follow either:

1. Implementation-first development, where AI agents receive tasks and generate solutions.

or:

2. Architecture-first development, where architectural artefacts guide human-driven implementation.

Neither approach fully addresses the need for a continuously synchronized knowledge layer connecting requirements, architecture, implementation, and testing. This research addresses this gap through an SSOT-centric lifecycle model.

II. METHODOLOGY

The proposed framework introduces the Single Source of Truth as the central intelligence layer of the software lifecycle. The SSOT is not treated as a documentation repository but as an evolving representation of system knowledge consumed by humans and AI agents. The framework begins with two possible initialization approaches. For new projects, engineering artefacts such as SRDs, technical evaluations, TDDs, ADRs, and C4 diagrams [1], are transformed into the SSOT. For existing projects, repository analysis is performed to discover system context from source code, dependencies, configuration files, tests, and deployment resources. The resulting SSOT contains structured knowledge representing product intent, architecture decisions, system behavior, engineering rules, security considerations, testing strategies, and deployment information.

The framework separates strategic planning from execution planning. The SSOT generates the implementation plan because the implementation strategy requires understanding of the complete system context. The task plan is generated from implementation strategies and governed by the SSOT. This separation prevents task-level execution details from becoming disconnected from architectural intent. This method allows for the generation of a system evolution view, which optimizes for system integrity and a construction view, which optimizes for completion respectively. This is possible because they are produced at different abstraction levels and through different reasoning processes. The source artefacts are the same, but the interpretation layer is different.

TABLE 1. SSOT-Centric framework layers

Layer	Role	Priority of Truth
SSOT	Governing Intelligence	1 st
Implementation Plan	Strategic Breakdown Execution	2 nd
Task Plan	Actionable Engineering Breakdown	3 rd
Agents	Execution Layer	

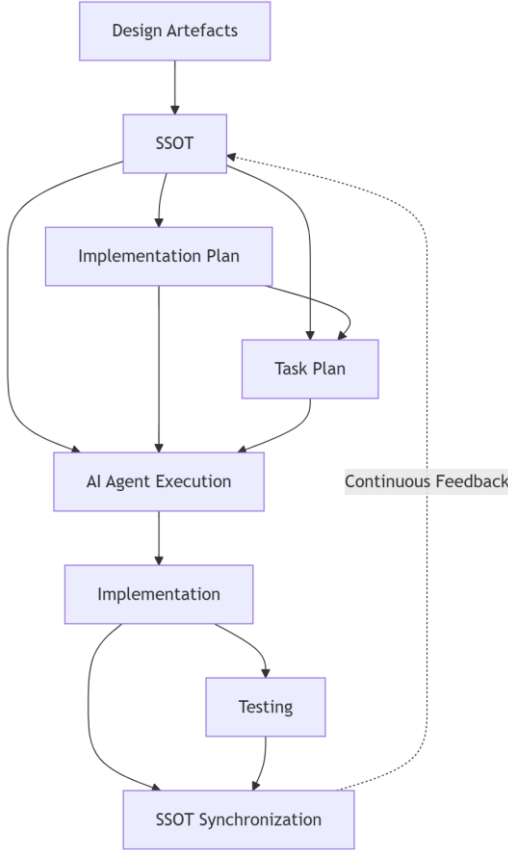


Figure 1. Proposed SSOT-Centric Agentic Software Engineering Framework

III. RESULTS AND DISCUSSION

The framework provides several advantages compared with conventional AI-assisted development approaches. First, it improves contextual understanding by providing AI agents with persistent system knowledge rather than isolated task descriptions. This allows agents to reason about system-wide implications before generating implementation changes.

Second, the framework reduces architectural drift by establishing synchronization between intended architecture and implemented reality. Instead of treating documentation as a final project artefact, the SSOT becomes an active engineering control mechanism.

Third, the separation between implementation planning and task planning creates a balance between strategic reasoning and execution precision. The implementation plan ensures that development follows system goals, while task plans provide actionable instructions for coding agents.

Finally, the framework supports scalable agent collaboration by enabling specialized agents to operate using a shared knowledge foundation. Architecture agents, implementation agents, testing agents, and security agents can coordinate through the SSOT rather than relying on isolated assumptions.

IV. CONCLUSION

This extended abstract paper presented an SSOT-Centric Agentic Software Engineering Framework actively being developed to support the emerging era of agentic software development. The framework introduces a Single Source of Truth as a knowledge governance layer connecting engineering artefacts, implementation strategies, task execution, and continuous system evolution. Unlike traditional implementation-first or architecture-first approaches, the proposed model combines architectural reasoning with execution capability by allowing SSOT-driven planning and design-driven task decomposition. This enables AI agents to operate with greater contextual awareness and reduces the risk of architectural inconsistency.

AI DECLARATION

I declare that AI tools was used to generate the figures, draft agent skills to demonstrate this workflow. ChatGPTv5 was also used to search for existing approaches and refinement of idea to support this work. Proper vetting was done to verify results. To that extent, the author takes full responsibility of the work and the contents therein.

REFERENCES

- [1] Brown, S. (2018). *The C4 model for visualising software architecture*. Retrieved from <https://c4model.com>.
- [2] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W. (2021). *Evaluating large language models trained on code*. arXiv. <https://arxiv.org/abs/2107.03374>.
- [3] Hou, X., Zhou, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A survey. *ACM Transactions on Software Engineering and Methodology*. <https://dl.acm.org/doi/10.1145/3695988>.